

程序设计实习大作业报告

——GPA Defence（保卫 GPA）趣味塔防游戏

王舒 王浩然 张曦月

2026 年 7 月 6 日

目 录

1. 程序功能介绍.....	1
1.1 游戏概述.....	1
1.2 核心玩法.....	1
1.3 交互设计.....	1
2. 项目各模块与类设计细节	2
2.1 项目整体架构	2
2.2 主窗口类（mainwindow）	2
2.3 怪物类（Enemy）	3
2.4 防御塔类（Tower）	4
2.5 子弹类（Bullet）	4
2.6 其他辅助类	5
2.7 界面窗口类	5
2.8 游戏资源管理	5
3. 小组成员分工情况	6
4. 项目总结与反思	7

1 程序功能介绍

1.1 游戏概述

本程序是一款基于 Qt6/C++ 开发的单人塔防策略游戏，灵感来源于经典塔防游戏《保卫萝卜》。游戏主题融入大学生生活趣味元素——玩家布置防御塔阻挡一波波“学术怪物”（ddl、pre、期中考试等）对“GPA 萝卜”的进攻，配合像素风格画面与背景音乐，呈现轻松有趣的游戏体验。

1.2 核心玩法

- **塔防机制**：怪物沿 S 形路径从左上角移动至右下角。玩家在路径两侧塔位上建造防御塔消灭怪物，防止其到达终点伤害萝卜。
- **经济系统**：初始 450 金币，击杀怪物获得 50-200 金币奖励，建造/升级消耗金币，拆除返还 50%。
- **障碍物系统**：部分塔位被障碍物占据，需先选中并命令防御塔清除后，方可腾出空位建塔。
- **多波次进攻**：最多 12 波怪物，强度随波次递增。最终波出现 BOSS“挂科（fail）”，击败 BOSS 且萝卜存活即为胜利。
- **难度选择**：简单（8 波）、中等（10 波）、困难（12 波），不同难度下怪物属性和波次数量不同。
- **辅助功能**：暂停/继续、二倍速加速、塔的升级与拆除。

1.3 交互设计

- **开始界面**：主菜单背景图 + “开始游戏”“规则说明”按钮。
- **难度选择**：半透明弹窗，绿/橙/红按钮对应简单/中等/困难。
- **倒计时**：3→2→1→GO 动画。
- **建塔流程**：点击空闲塔位 → 弹出四种塔选择菜单 → 点击建造。
- **塔交互**：点击已建塔弹出“升级”“拆除”按钮。
- **障碍物交互**：点击障碍物标记为攻击目标，射程内的塔优先攻击。
- **结算界面**：胜利/失败图片 + “重新开始”“返回主菜单”按钮。

2 项目各模块与类设计细节

2.1 项目整体架构

项目采用 Qt6 框架（C++11），CMake 构建。共包含 10 个核心类和 3 个窗口界面，通过 Qt 的信号-槽机制和 QTimer 定时器驱动游戏逻辑。以下为类的组织关系：

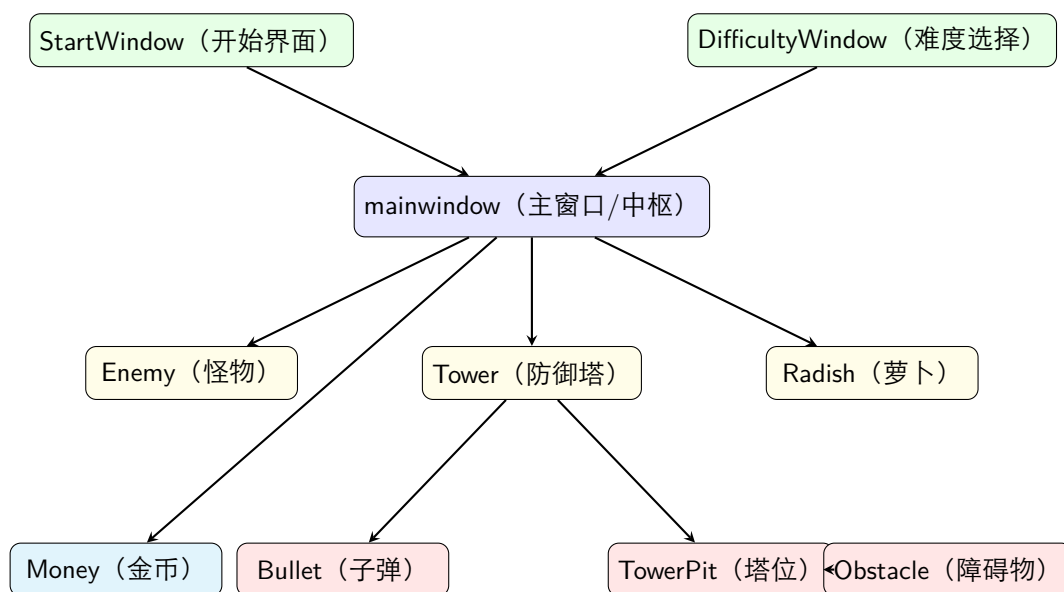


图 1 项目核心类关系图

2.2 主窗口类 (mainwindow)

mainwindow 继承 QWidget，是整个游戏的控制中枢。

类声明 (关键部分):

```

class mainwindow : public QWidget {
    Q_OBJECT
public:
    mainwindow(QWidget *parent = nullptr);
protected:
    void paintEvent(QPaintEvent*) override;
    void timerEvent(QTimerEvent*) override;
    void mousePressEvent(QMouseEvent*) override;
private:
    Ui::mainwindow* ui;    QMediaPlayer* bgmPlayer;
    QList<Enemy*> enemies;  QList<Tower*> towers;
    Radish* radish;    Money* money;
    int currentWave; int totalWaves; int difficulty;
    QList<TowerPit*> towerPits; QList<Obstacle*> obstacles;
    bool isGameOver; bool isPaused; bool isVictory;
    Tower* selectedTower; Obstacle* selectedObstacle;
    void generateMonster(int type);
    bool isEnemyAtEnd(Enemy* enemy);};
  
```

核心职责:

- 管理游戏状态 (未开始/进行中/暂停/胜利/失败) 与所有实体生命周期
- 通过 timerEvent 驱动游戏主循环: 每 100ms (二倍速 50ms) 触发一次, 依次完成波次怪物生成、怪物移动与碰撞检测、防御塔攻击寻敌与子弹发射、子弹飞行与伤害计算、障碍物摧毁检测、萝卜血量检查与胜负判定
- 处理鼠标点击事件, 按优先级链响应: 结算按钮 → 全局按钮 → 升级/拆除 → 建塔菜单 → 障碍物选择 → 塔位点击 → 塔选择

- 控制波次生成：每波持续 20–40 秒，按预设时间间隔产出不同组合的怪物，血量加成随波次递增 (0%–300%)，受难度系数调节

2.3 怪物类 (Enemy)

Enemy 类封装所有敌方单位，包含 6 种子类型。

类声明 (关键部分)：

```
class Enemy {
public:
    enum Type { Monster1, Monster2, Monster3,
                Monster4, Monster5, MonsterBoss };
    Enemy(Type type);
    void move(); void draw(QPainter*);
    void takeDamage(int damage);
    void applySlow(double factor, int durationMs);
    int getHealth() const; Type getType() const;
    QPointF getPosition() const; double getTraveledDistance() const;
    void addHealth(int amount);
private:
    Type type; QPointF position; double speed;
    int health, maxHealth; QPixmap enemyPixmap;
    QVector<QPointF> pathPoints; int currentPathIndex;
    bool m_isSlowed; QTime m_slowEndTime;};
```

表 1: 怪物属性一览

类型	名称	速度	生命值	击杀金币
Monster1	ddl (截止日期)	4.0	500	50
Monster2	pre (课堂展示)	6.0	500	80
Monster3	mid_term (期中)	4.0	600	80
Monster4	pre (演讲)	3.5	600	80
Monster5	ddl_1 (截止日)	4.0	600	80
MonsterBoss	fail (挂科)	2.0	8000	200

核心设计:move() 沿 6 个关键点构成的 S 形折线逐段移动,到达新目标点时切换方向;applySlow() 实现减速 (MUSIC 塔子弹触发, 50% 减速持续 3 秒,到期自动恢复原速);addHealth() 用于波次血量动态加成;draw() 中绘制血条 (绿 → 橙 → 红渐变) 并支持双帧精灵动画和被减速时的蓝色色调渲染。

2.4 防御塔类 (Tower)

Tower 类封装四种防御塔,采用枚举 Type { COFFEE, MUSIC, AI, TEACHER } 区分类型。

类声明 (关键部分)：

```
class Tower {
public:
    enum Type { COFFEE, MUSIC, AI, TEACHER };
    Tower(Type type, const QPoint& pos);
```

```

void attack(const QList<Enemy*>& enemies);
void attackObstacle(Obstacle* obstacle);
void updateBullets(); void upgrade();
int getRange() const; int getUpgradeCost() const;
int getSellPrice() const;
static int getCost(Type type);
static void setGameSpeed(int speed);
private:
    Type towerType; QPoint position;
    int range, damage, level, upgradeCost;
    std::vector<Bullet*> bullets; QTimer lastShotTime;
    static int s_gameSpeed;};

```

表 2: 防御塔属性一览

类型	名称	费用	射程	伤害	攻速	效果
COFFEE	咖啡续命	100	200	40	0.5s	旋转动画
MUSIC	音乐放松	120	200	48	1.0s	减速 50%
AI	AI 辅助	160	240	84	1.5s	—
TEACHER	老师指导	180	240	84	2.0s	—

关键逻辑: `attack()` 遍历所有怪物, 计算曼哈顿距离判断是否在射程内, 优先选择路径行进最远 (`getTraveledDistance()` 最大) 的目标; 按攻击间隔 (受 `s_gameSpeed` 静态变量缩放) 创建 `Bullet` 实例。`upgrade()` 提升等级、伤害 +36、射程 +30。`getSellPrice()` 返还基础造价的 50%。`COFFEE` 塔在绘制时使用 `QPainter` 的 `translate/rotate` 实现旋转效果。

2.5 子弹类 (Bullet)

`Bullet` 类封装弹道逻辑, 提供两个构造函数重载: `Bullet(QPoint, Enemy*, int, QString)` 和 `Bullet(QPoint, Obstacle*, int, QString)`。每帧调用 `move()` 向目标移动 20 px, 到达后 (曼哈顿距离 $\leq$ 速度) 调用 `checkAndApplyDamage()` 造成伤害并标记 `m_isReachedTarget`。`MUSIC` 塔子弹设置 `m_shouldSlow = true`, 命中时对怪物调用 `applySlow(0.5, 3000)`。四种塔分别使用不同子弹图片 (`bullet_late_night_cutout.png`、`bullet.png`、`ai_bullet_cutout.png`、`bullet_seniors_cutout.png`)。

2.6 其他辅助类

- **Radish (萝卜)**: 位于 (750, 600), 初始 10 HP。`takeDamage(double)` 接受不同伤害: `Monster1` 造成 0.5 点, `Monster2-5` 造成 1.0 点, `Boss` 造成 10.0 点。血量降低时自动切换显示 `gpaa_{hp}.png` (如 `gpaa_10`→`gpaa_1`), 归零触发游戏失败。
- **Money (金币)**: 封装经济系统, 初始 450。提供 `canAfford/spend/earn/reset` 接口, 主界面通过 `QPainter` 实时绘制余额。
- **TowerPit (塔位)**: 静态方法 `createTowerPits()` 按 5 行布局生成 43 个塔位, 维护三个布尔状态: `hasTower/hasObstacle`。点击空闲塔位时弹出建塔菜单。
- **Obstacle (障碍物)**: 10 个障碍物分布在特定塔位 (`index 9, 15, 17, 19, 20, 22, 25, 33, 34, 38`), 各 300 HP。支持选中状态, 被摧毁后自动调用 `TowerPit::setObstacle(nullptr)` 释放塔位。

- **Button (按钮)**: 通用 UI 组件, 维护 `isActive` 状态切换 `normal/hover` 两张图片, 用于暂停、加速等全局控制。

2.7 界面窗口类

- **StartWindow**: 继承 `QWidget`, 固定 `1200×800`。通过 `paintEvent` 绘制 `start_page.png` 背景, 在 `(413, 349)` 和 `(1000, 700)` 两个区域检测鼠标点击, 分别发射 `startGame()` 和 `showRules()` 信号。
- **DifficultyWindow**: 半透明深色覆盖层 (`rgba(0,0,0,180)`), 三个彩色按钮 (绿/橙/红) 分别对应简单/中等/困难, 点击后发射 `difficultySelected(int)` 信号。

2.8 游戏资源管理

约 140 张图片与 1 首背景音乐通过 Qt 资源系统 (`resources.qrc`) 统一管理, 在代码中以 `qrc:/images/...` 和 `qrc:/sounds/...` 路径引用。背景音乐使用 `QMediaPlayer + QAudioOutput` 循环播放, 音量 0.01。

3 小组成员分工情况

表 3: 小组成员分工明细

成员	主要职责	具体工作内容
全体成员	选题与方向确定	共同商议游戏赛道方向和选题创意, 将大学生生活元素融入游戏机制
	代码框架设计	共同设计核心玩法、波次系统、经济平衡, 确定类的划分与接口规范
王舒	素材加工与 UI 编码	负责图片后期处理、格式统一、尺寸适配, 参与 UI 界面代码编写与资源文件 (<code>qrc</code>) 配置
	报告撰写与项目统筹	承担大作业报告的撰写工作, 协调项目整体进度
	代码测试与调试	参与游戏测试与代码调试, 协助发现和定位程序问题
王浩然	核心逻辑编码	承担游戏核心逻辑的编码工作, 完成 <code>Enemy/Tower/Bullet</code> 等关键类的具体实现
	游戏逻辑调试与优化	负责波次逻辑、怪物 AI、塔攻击机制等核心逻辑的调试优化
	构建配置与集成	负责 <code>CMake</code> 构建配置、跨平台测试与素材集成
张曦月	视觉素材创作	负责游戏全部视觉素材的原创绘制与收集整理
	界面编码与仓库展示	参与界面布局代码编写, 搭建 <code>GitHub</code> 仓库并完成网页排版展示
	测试与反馈	参与游戏测试, 根据反馈调整美术资源与程序问题

分工协作流程：

1. **共同商议阶段**：三人共同讨论确定塔防游戏赛道与选题方向，融入大学生活趣味元素（ddl、pre、期中考试等）。一同设计核心玩法和代码框架，确保后续接口清晰。
2. **并行开发阶段**：张曦月负责视觉素材创作与界面设计；王舒对素材进行加工处理，参与 UI 层与资源文件的配置；王浩然承担核心游戏逻辑的编码实现。
3. **整合测试阶段**：王浩然负责素材集成与联调测试；王舒和张曦月共同参与测试反馈与代码调试，根据反馈调整程序与美术资源。
4. **文档收尾阶段**：张曦月搭建 GitHub 仓库并完成网页排版展示；王浩然提供项目代码整体说明；王舒撰写大作业报告。

4 项目总结与反思

4.1 项目成果

我们成功实现了一款功能完整的塔防游戏，具备以下亮点：

- **完整的游戏循环**：从菜单到结算形成闭环体验。
- **丰富的策略深度**：四种塔各有特色（高速输出型、减速控制型、均衡型、高伤型），玩家需合理搭配。障碍物清除机制进一步增加战略层次。
- **生动的主题包装**：以学术压力元素为怪物、GPA 为血量指示、咖啡/音乐/AI/老师为防御塔概念，代入感强。
- **良好的用户体验**：支持难度选择、二倍速、暂停、升级拆除等辅助功能。

4.2 技术收获

1. **Qt 图形编程**：深入掌握 QPainter 自定义绘制（精灵渲染、血条、旋转动画）和 QTimer 驱动的游戏主循环模式。
2. **面向对象设计**：将每个游戏实体抽象为独立类，通过类协作实现复杂逻辑，结构清晰、易于维护扩展。
3. **游戏开发概念**：实践了游戏主循环、碰撞检测、波次管理、经济平衡、状态机等核心概念。
4. **团队协作**：学习了任务分配、接口规范统一、联调测试等软件工程实践。
5. **CMake 构建**：掌握了 Qt 模块查找链接、AUTOMOC/AUTOUI/AUTORCC 自动处理、资源管理。

4.3 不足与改进方向

1. **波次系统硬编码**：波次逻辑在 timerEvent 中用大量 if-else 实现，应数据化配置（如 JSON）。
2. **缺少存档功能**：不支持中途存档读档，影响长时游戏体验。
3. **地图单一**：仅一张地图，可扩展多张不同布局地图提升可重玩性。
4. **怪物 AI 简单**：沿固定路径移动，无寻路避障等高级行为；塔的目标选择策略也较基础。
5. **音效不足**：仅有背景音乐，缺少攻击/死亡/胜利/失败音效。
6. **代码重复**：restartGame() 与 returnToStart() 中存在重复逻辑，可抽取公共方法。

4.4 心得体会

本次大作业是一次有意义的实践经历。从选题讨论、框架设计，到编码实现、素材制作，再到整合测试和文档撰写，我们完整经历了一个小型游戏项目的开发周期。

在技术层面，我们将课堂上所学的 C++ 面向对象编程、Qt 框架等知识应用于实际项目中，加深了对理论的理解。在团队协作层面，我们体会到分工合作的重要性——合理的任务分配和及时的

沟通协调是项目顺利推进的关键。

感谢王浩然同学承担了大量编码调试工作，使设计方案高质量落地；感谢张曦月同学精心制作的美术素材，为游戏注入生动表现力。这次合作让我们深刻体会到：成功的项目需要技术与艺术的紧密结合，也需要团队成员之间的相互信任与支持。